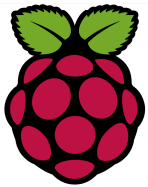


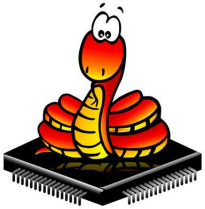


# Atelier Raspi

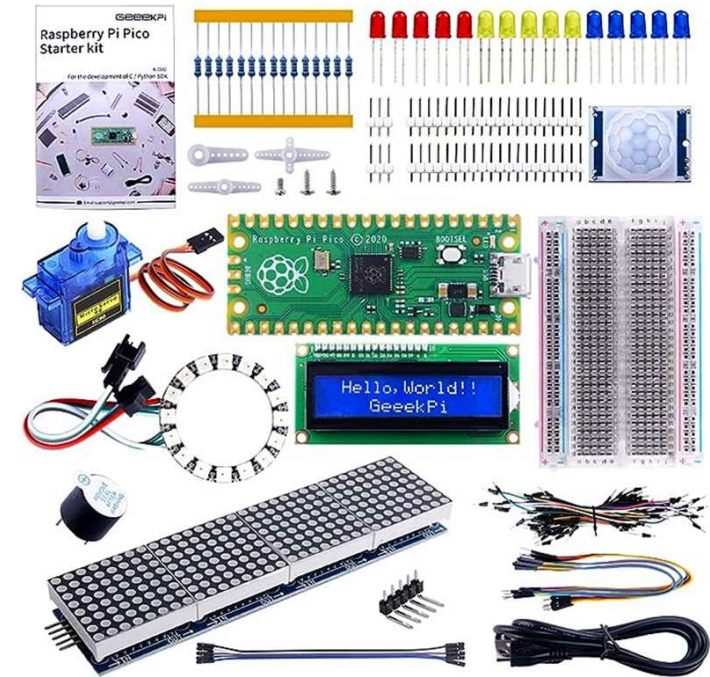
## Atelier N°12 La mini serre instrumentée DHT11/22, KY40



Logo du Raspberry Pico



Logo du MicroPython

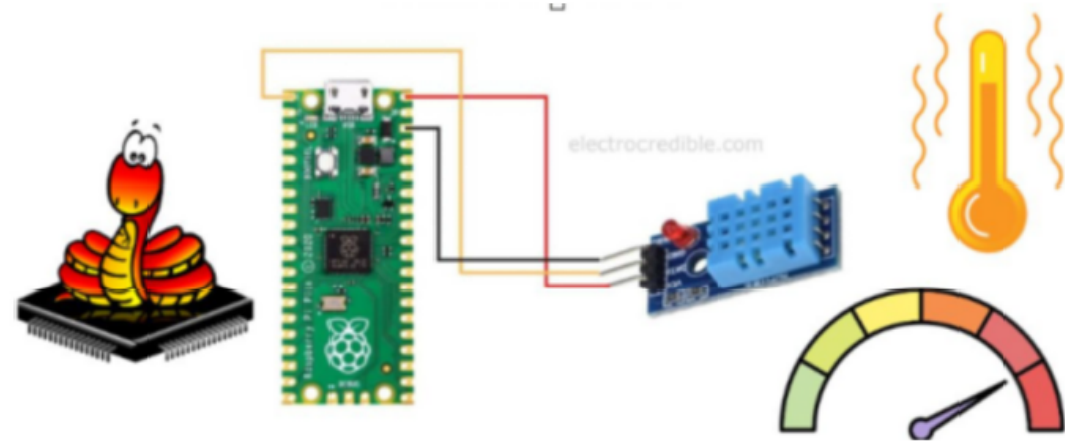


L'atelier a pour valeurs, le partage, l'aide, la formation, le faire et construire ensemble à partir de l'expérience des participants

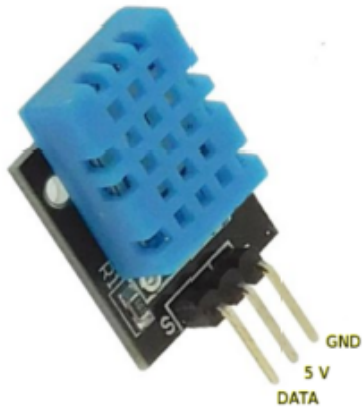
# Atelier N°12 La mini serre instrumentée

## Capteurs DHT11/22, KY40

- Etape 1 Le capteur DHT11/22
- Etape 2 Le capteur de température
- Etape 3 Le capteur d'humidité
- Etape 4 Le protocole « one wire »
- Etape 5 Câblage du capteur
- Etape 6 Affichage des données du capteur
- Etape 7 Rajout des 4 leds, câblage
- Etape 8 Cahier des charges du programme
- Etape 9 Le programme lecture capteur et affichage leds 1/2
- Etape 10 L'encodeur KY40
- Etape 11 Construction du KY40
- Etape 12 Les signaux du KY40
- Etape 13 Le schéma électrique
- Etape 14 Le programme 1/2
- Etape 15 Le programme 2/2
- GI1 Glossaire informatique1
- GI2 Glossaire informatique2
- GI3 Glossaire informatique3
- GI4 Glossaire informatique4
- GI5 Glossaire informatique5
- GI6 Glossaire informatique6
- GE1 Glossaire électronique



# A12E1 La mini serre instrumentée le capteur DHT11/22



**DHT11**

Les capteurs DHT11 et DHT22 mesurent deux choses :

- la température de l'air 🌞 ❄️
- l'humidité relative 💧 (la quantité de vapeur d'eau présente dans l'air)

Ces capteurs sont souvent utilisés dans les projets électroniques et les expériences scientifiques pour surveiller les conditions météo ou l'environnement d'une pièce.



**DHT22**

Le capteur DHT22 est très similaire au DHT11, mais il offre une meilleure précision de mesure ( $\pm 0,5$  °C) et il peut être alimenté avec une tension de 3.3V ou 5V.

| Caractéristiques    | DHT11        | DHT22              |
|---------------------|--------------|--------------------|
| Température mesurée | 0 à 50 °C    | -40 à +80 °C       |
| Humidité mesurée    | 20 à 80 %    | 0 à 100 %          |
| Précision           | Moyenne      | Élevée             |
| Prix                | Moins cher 🍀 | Un peu plus cher 🍂 |

Il faut faire attention car le câblage du module DHT22 est différent du DHT11. Les broches GND et DATA sont inversées sur ces deux modules.

# A12E2 La mini serre instrumentée DHT11/22

## Le capteur de température

Pour mesurer la température, le DHT11 utilise un capteur de température NTC (*Negative Temperature Coefficient*), aussi appelé **thermistance**.

Ce capteur est monté à l'intérieur du boîtier en plastique du DHT11.

Un capteur NTC est un capteur à résistance variable :

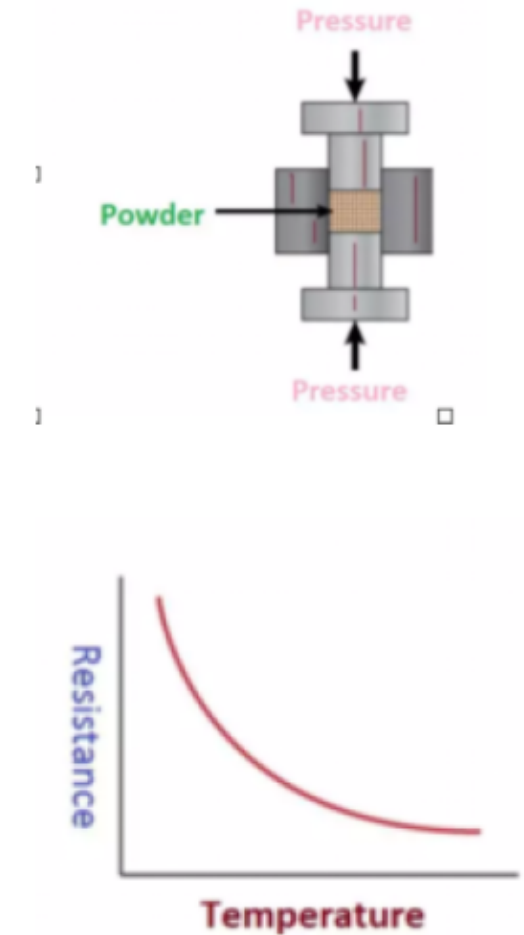
👉 sa résistance diminue quand la température augmente.

Les thermistances sont fabriquées à partir de matériaux semi-conducteurs (comme la céramique ou certains polymères).

Elles ont la particularité de faire varier fortement leur résistance même pour de petites variations de température — ce qui les rend très sensibles et précises pour ce type de mesure.

Voici un graphique typique qui montre la relation entre la température et la résistance pour le capteur DHT11 :

📉 Quand la température augmente, la résistance du capteur diminue.



## A12E3 La mini serre instrumentée DHT11/22

### Le capteur d'humidité

Pour mesurer l'humidité, le DHT11 utilise un capteur capacitif d'humidité.

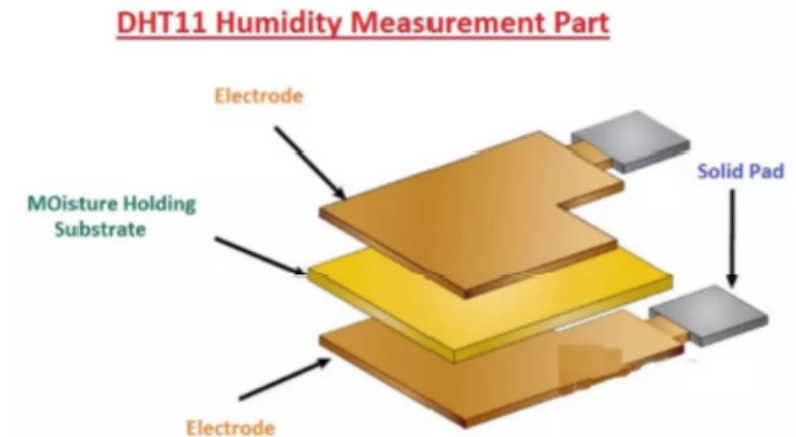
Ce capteur est constitué de deux électrodes séparées par un matériau appelé substrat.

Le substrat a pour rôle de retenir l'humidité à sa surface.

Lorsque la quantité d'humidité dans l'air change, le substrat absorbe ou libère de l'eau, ce qui modifie la résistance électrique entre les deux électrodes.

Cette variation de résistance est ensuite convertie en valeur d'humidité relative grâce à un coefficient d'humidité enregistré dans la mémoire du capteur.

Le DHT11 peut ainsi fournir la valeur finale de l'humidité relative de l'air autour de lui.





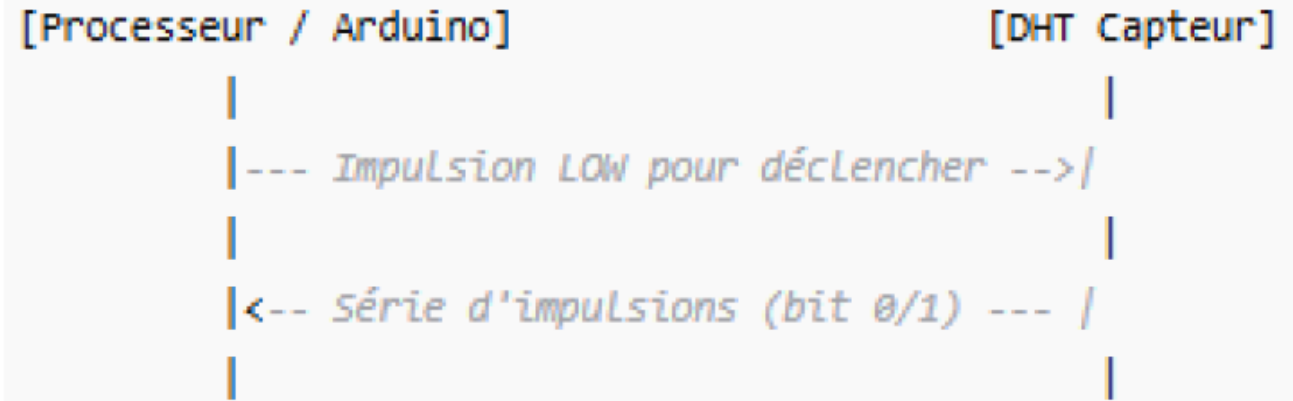
# A12E4 La mini serre instrumentée DHT11/22

## Le protocole « one wire »

Pour lire l'humidité, **le processeur enverra une impulsion basse** sur la ligne data pour déclencher le transfert **et lira ensuite les impulsions envoyées par le DHT.**

Si l'impulsion basse est plus longue que l'impulsion haute qui suit il s'agit d'un bit à 1 sinon, il s'agit d'un bit à 0.

Le composant enverra l'humidité (8 bits partie entière et 8 bits partie décimale) et la température (même format) suivi d'une somme de contrôle.



DHT11 sends the 40Bit serial data in the below format:

8-Bit Humidity(Integral)

8-Bit Humidity(Decimal)

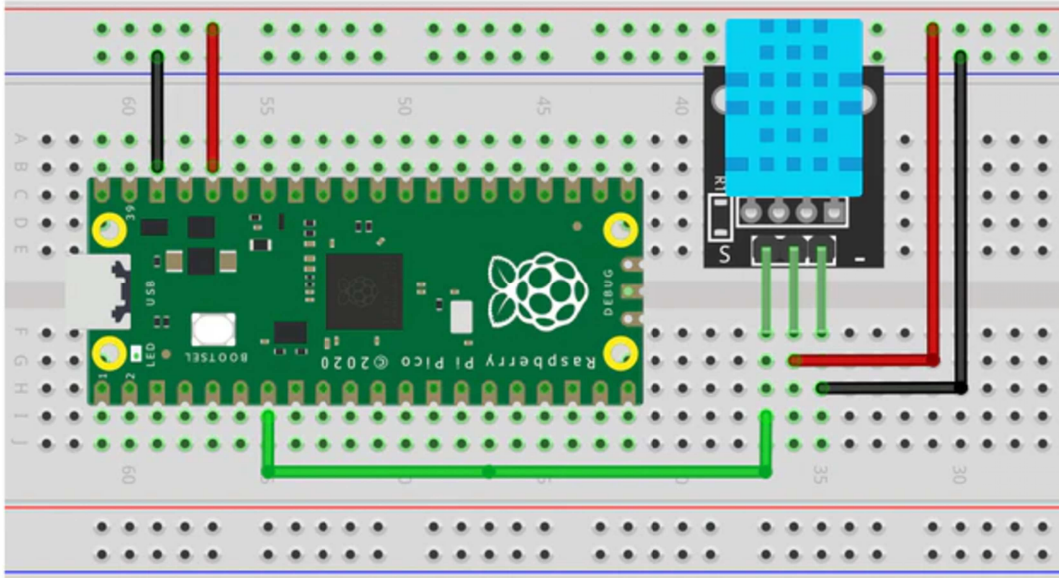
8-Bit Temperature(Integral)

8-Bit Temperature(Decimal)

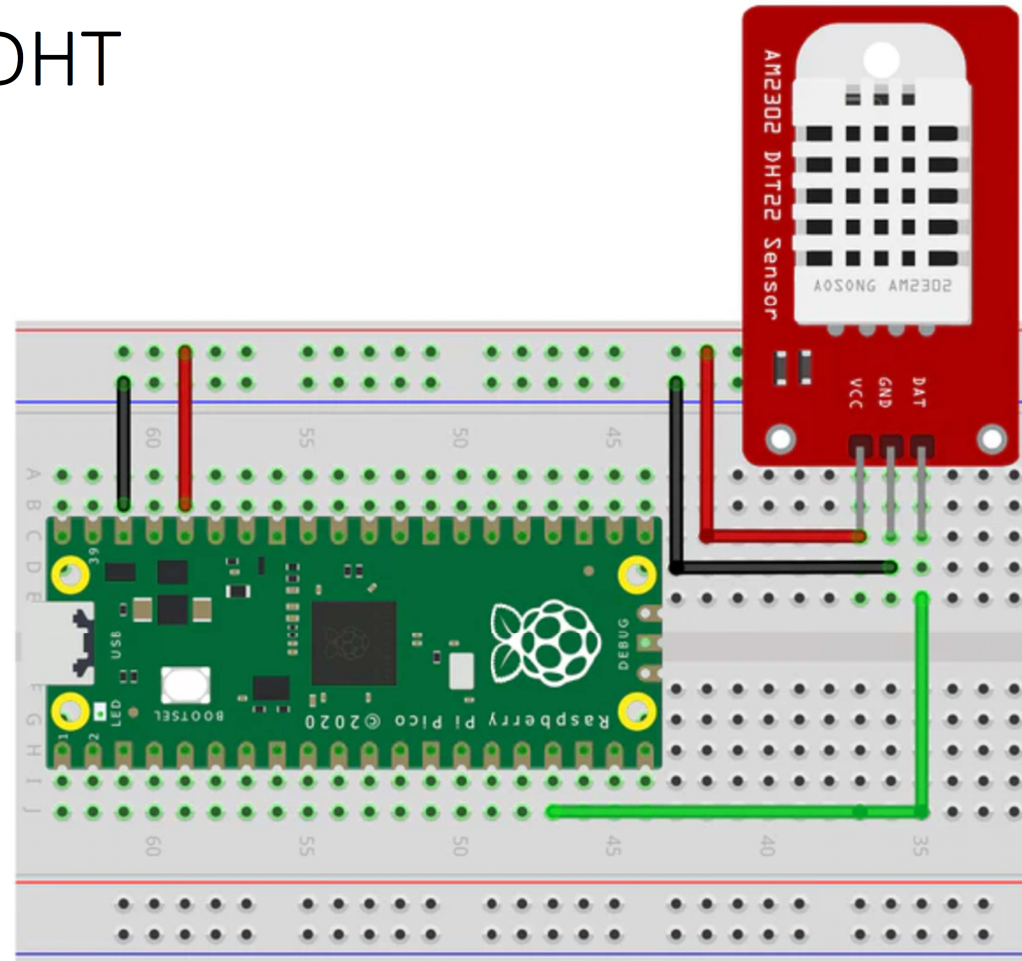
8-Bit Checksum

If DHT11 is sending the correct data, then it must send an 8-bit Checksum data at the end.

# A12E5 Exercice 1 : Câblage du DHT



Pin de lecture du capteur: GP5



Pin GP13 **ATTENTION**

Suivant le modèle de dht22, l'alimentation + et- est inversée

## A12E6 Exercice 2 : Affichage des données

```
from machine import Pin
from time import sleep
import dht

DHT11 = dht.DHT11(Pin(22))

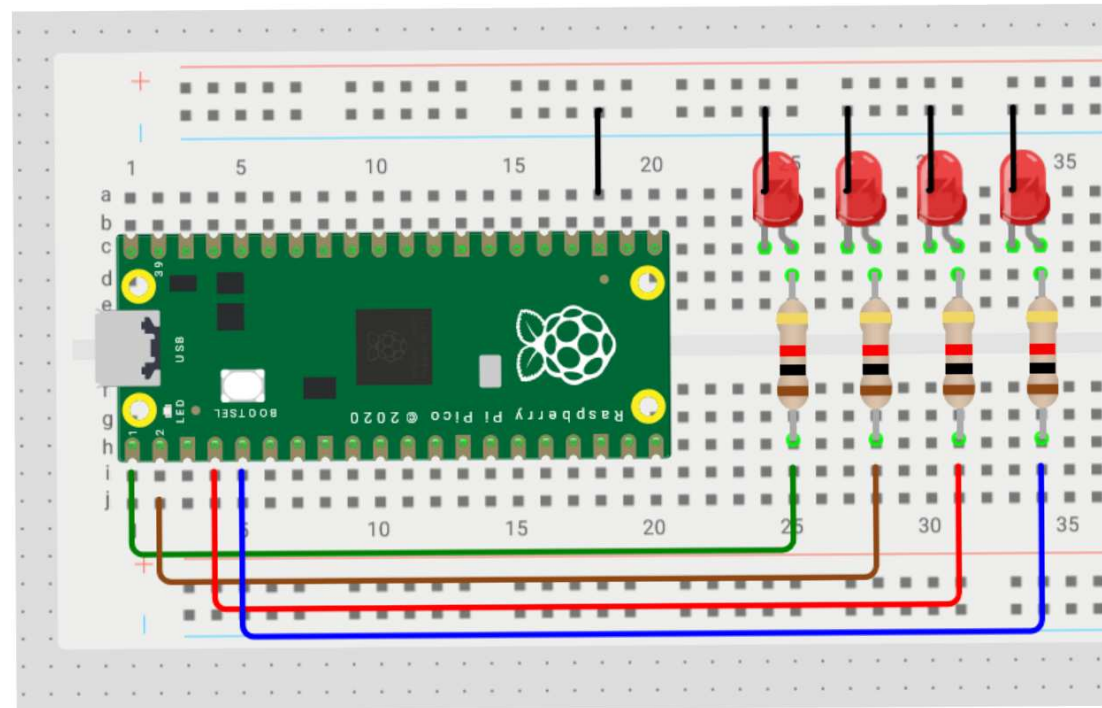
while True:
    DHT11.measure()
    print(f"Temperature : {DHT11.temperature():.1f}")
    print(f"Humidite      : {DHT11.humidity():.1f}")
    sleep(1)
```

Le Pin de lecture sera différent suivant le choix lors du câblage



# A12E7 Exercice 3 : Rajouter les 4 leds

| LED   | Broche | GPIO |
|-------|--------|------|
| led_0 | 1      | 0    |
| led_1 | 2      | 1    |
| led_2 | 4      | 2    |
| led_3 | 5      | 3    |



# A12E8 Exercice 4 : Coder la T° sur 4 LEDs

## Objectif :

Importer les modules nécessaires

Déclarer le capteur DHT11 et les 4 LEDs

Déclarer une liste contenant les 4 leds

Parcourir la liste de led pour les éteindre

Boucle infinie :

- Lecture du capteur

- Si temperature < 21, éteindre les leds

- Si temperature < 22, allumer 1 led

- Si temperature < 23, allumer 2 leds

- Si temperature < 24, allumer 3 leds

- Sinon allumer toutes les leds

- Attendre 2 secondes

# A12E9 Programme T° sur 4 LEDs

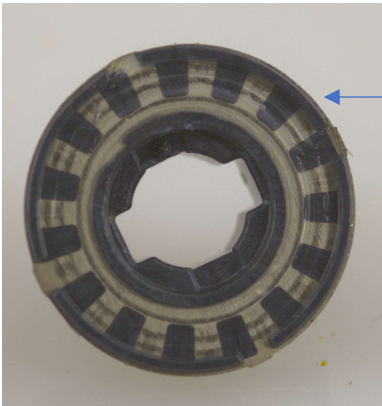
```
1 from machine import Pin
2 from time import sleep
3 import dht
4
5 #Declaration du capteur
6 DHT11 = dht.DHT11(Pin(22))
7
8 #Declaration des leds
9 led_0 = Pin(6, Pin.OUT)
10 led_1 = Pin(7, Pin.OUT)
11 led_2 = Pin(8, Pin.OUT)
12 led_3 = Pin(9, Pin.OUT)
13
14 #Declaration des leds sous forme de liste
15 LEDS = [led_0, led_1, led_2, led_3]
16
17 #Eteindre toutes les leds
18 for item in LEDS:
19     item.value(0)
20
21
22 # --- Boucle principale ---
23 while True:
24     #On lit le capteur
25     DHT11.measure()
26
27     #Allumer les 3 premieres leds, eteindre la derniere
28     LEDS[0].value(1)
29     LEDS[1].value(1)
30     LEDS[2].value(1)
31     LEDS[3].value(0)
32
33     else:
34         #Allumer toutes les leds
35         for i in range(0, 4):
36             LEDS[i].value(1)
37
38     #Attendre 2 secondes avant de relire la température
39     sleep(2)
```

```
26 temp = DHT11.temperature()
27 hum = DHT11.humidity()
28 #On affiche sur la console pour vérifier la valeur
29 print("Température:", temp, "°C | Humidité:", hum, "%")
30
31 if temp < 21:
32     #Eteindre toutes les leds
33     for i in range(0, 4):
34         LEDS[i].value(0)
35
36 elif temp < 22:
37     #Allumer la premiere led, eteindre les autres
38     LEDS[0].value(1)
39     LEDS[1].value(0)
40     LEDS[2].value(0)
41     LEDS[3].value(0)
42
43 elif temp < 23:
44     #Allumer les 2 premieres leds, eteindre les autres
45     LEDS[0].value(1)
46     LEDS[1].value(1)
47     LEDS[2].value(0)
48     LEDS[3].value(0)
49
50 elif temp < 24:
```

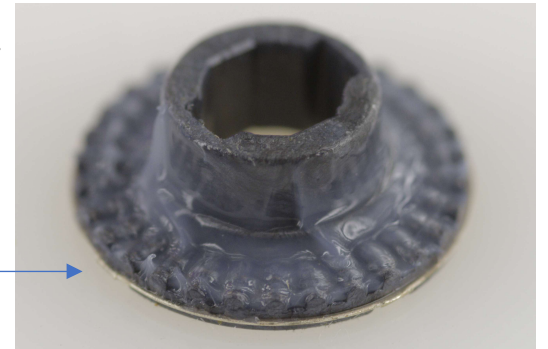
## A11E10 La mini serre instrumentée, l'encodeur KY40



Le KY40 est un encodeur rotatif est un dispositif mécanique constitué d'un bouton rotatif, qui lorsqu'on le tourne, actionne 2 contacts électriques l'un après l'autre. Lorsque l'on tourne le bouton dans les deux sens, les contacts électriques donnent des signaux carrés l'un après l'autre, on peut en déduire le sens de rotation du bouton. Comme les contacts sont répartis régulièrement sur la circonférence, on peut en déduire l'angle de rotation du bouton si l'on compte le nombre de crans. Il y a aussi une fonction bouton poussoir lorsque l'on appuie dessus.



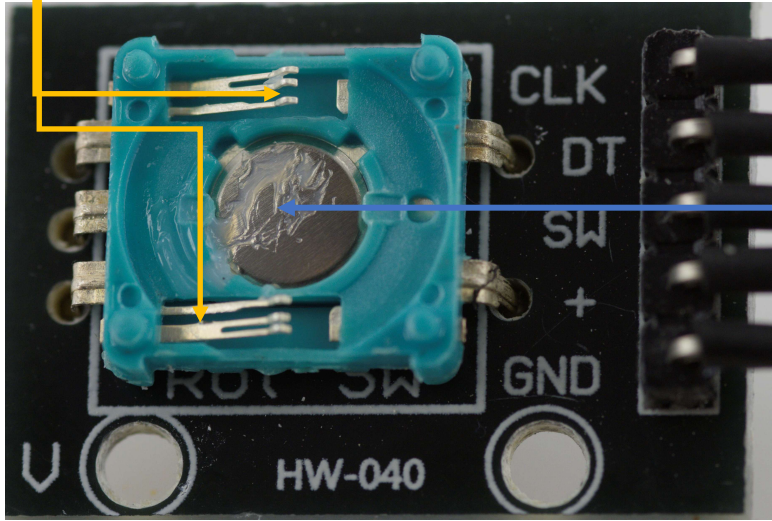
Sur l'axe de rotation est fixé un disque, aussi appelé « roue encodeuse »  
Ce disque est conducteur par endroits, et isolant à d'autres.  
A l'arrière il y a des crans qui permettent de garder la rotation dans une position.



# A12E11 La mini serre instrumentée

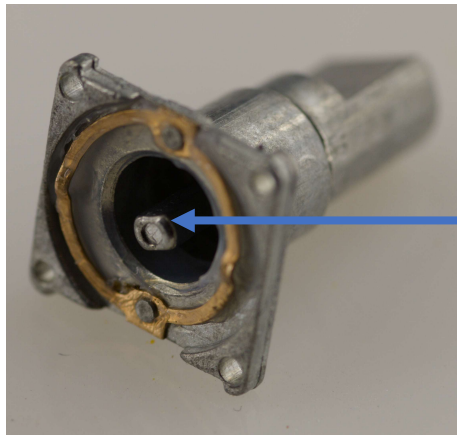
## Construction du KY40

Les lamelles de contacts conductrices (qui sont en fait reliées aux pins CLK, DT, et COMMUN, notées sur le PCB)



Sur le disque viennent principalement reposer 2 contacts, qui sont disposés à des endroits particuliers.  
Et ces contacts sont « ouverts » ou « fermés » selon s'ils se trouvent dans une zone isolante ou conductrice du disque.

Bouton poussoir central



Axe de rotation du bouton

Fixation de l'ensemble

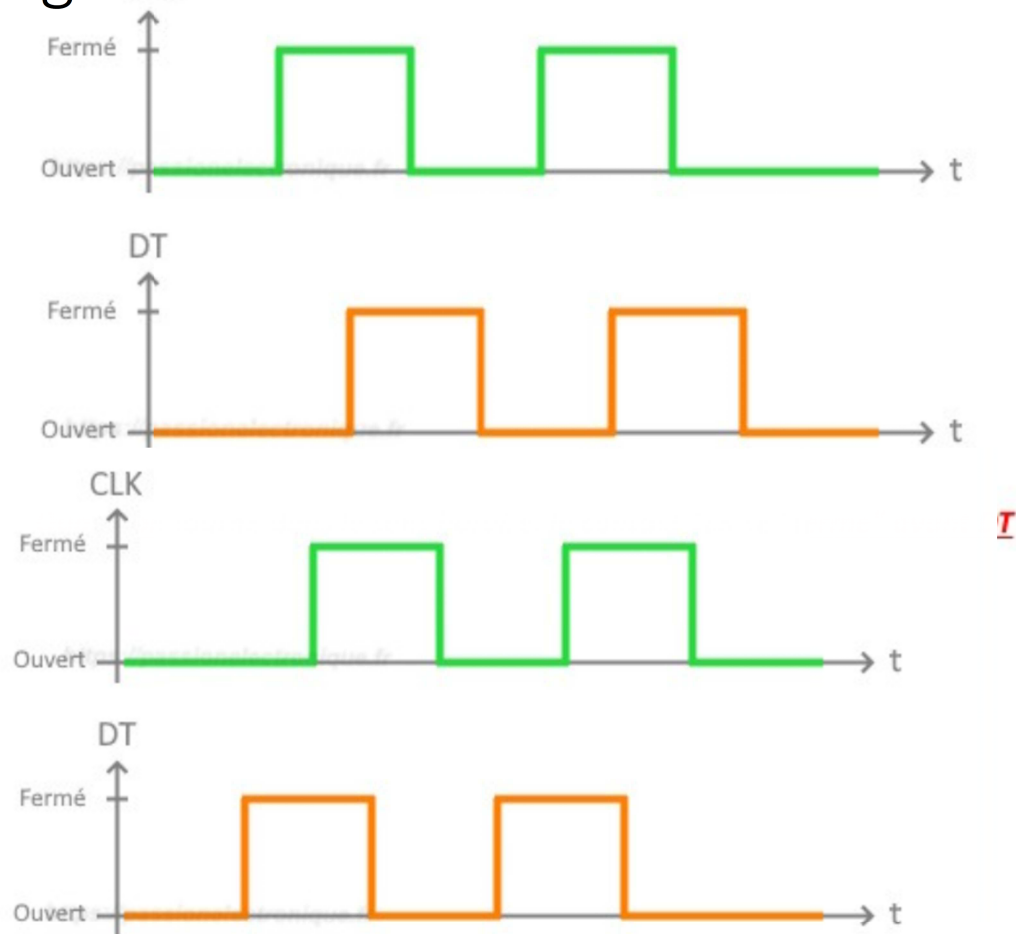
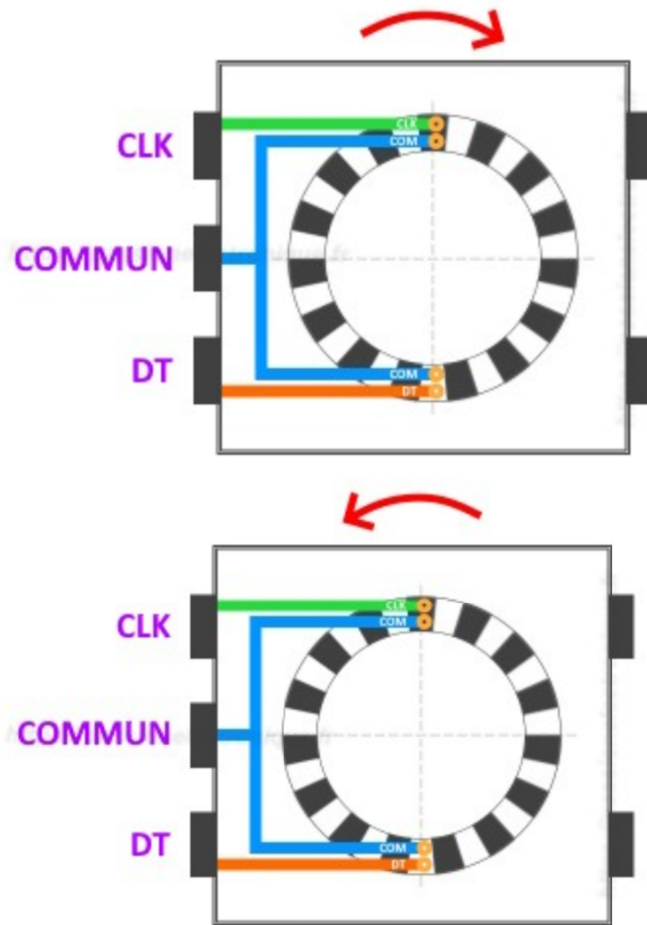


Coupleur de l'axe et du Bouton poussoir



# A12E12 La mini serre instrumentée

## Les signaux du KY40

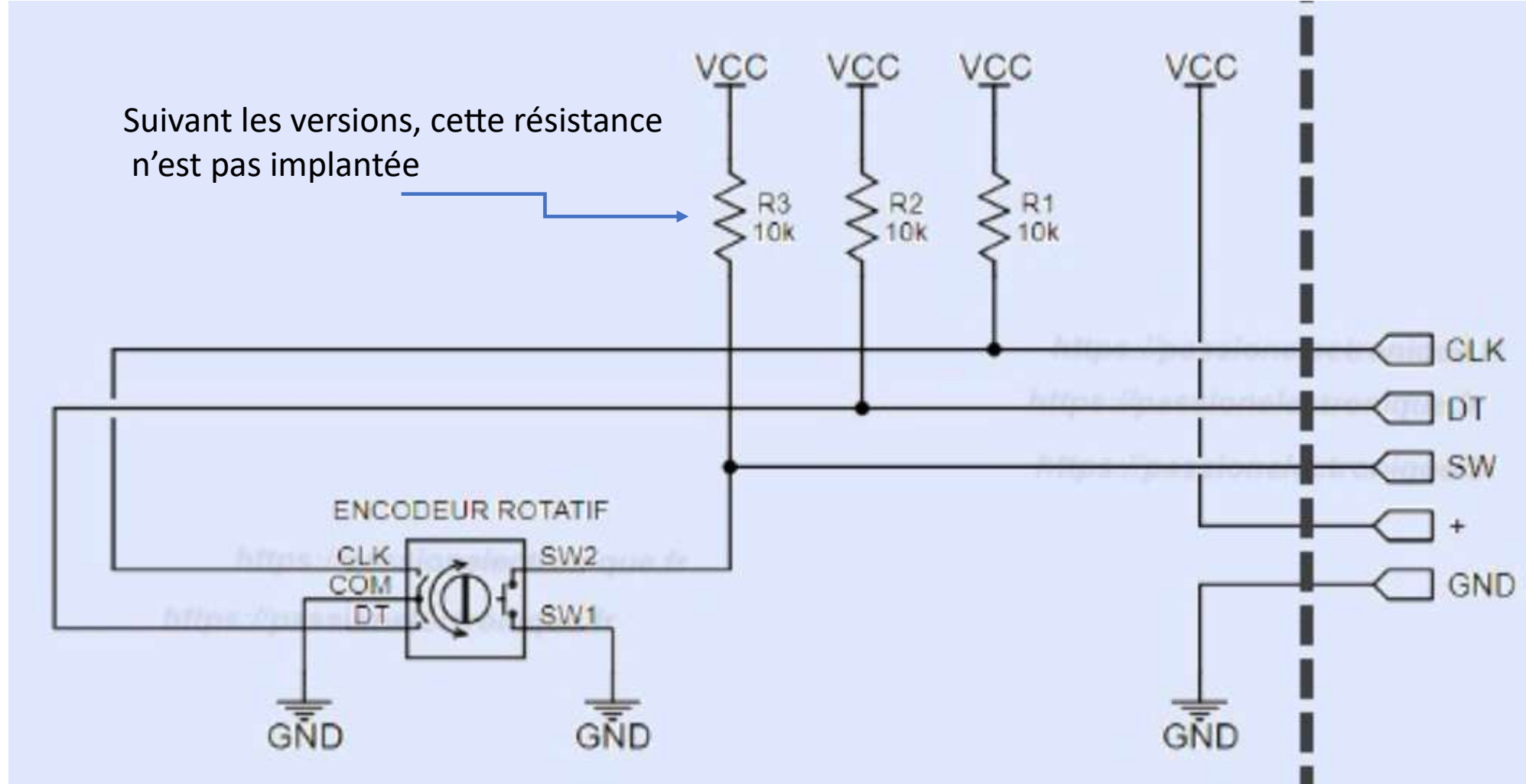


Quand on tourne dans le sens anti-horaire, le contact CLK se "ferme" après DT

L'encodeur rotatif incrémental est donc un bouton rotatif, qui ouvre et ferme ses contacts à des moments précis, et ce, différemment selon dans quel sens on tourne le bouton.

# A12E13 La mini serre instrumentée KY40

## Le schéma électrique



# A12E14 La mini serre instrumentée KY40

## Le programme 1/2

```
1 from machine import Pin
2 from rotary_irq_rp2 import RotaryIRQ # importation de RotaryIRQ qui est dans le fichier rotary_ir
3 import time
4 # initialisation des leds
5 LED0=Pin(0, Pin.OUT)
6 LED0.value(0)
7 LED1=Pin(1, Pin.OUT)
8 LED1.value(0)
9 LED2=Pin(2, Pin.OUT)
10 LED2.value(0)
11 LED3=Pin(3, Pin.OUT)
12 LED3.value(0)
13 LED=[LED0,LED1,LED2,LED3] #creation de la liste des leds
14
15 #Declaration de l'encoder
16 ENCODER = RotaryIRQ(pin_num_clk=6,pin_num_dt=7,reverse=False,
17                     incr=1,range_mode=RotaryIRQ.RANGE_UNBOUNDED,
18                     pull_up=True,half_step=False)
19
20 val_old = ENCODER.value() # initialisation de la variable val_old avec la valeur qu'il y a
21                          #dans ENCODER.value()
22
23 def lecture_encoder(enc): # la fonction lecture de l'encodeur
24     global val_old
25     val_new = enc.value()
```

# A12E15 La mini serre instrumentée KY40

## Le programme 2/2

```
26     if val_old != val_new: #si la valeur ancienne de l'encodeur est différente de celle qui
27                           #est actuellement, on incremente ou decremente un compteur
28         for led in LED:  # on met toutes les leds à 0 donc éteintes
29             led.value(0)
30     I= val_new % 4        # on calcule le reste de la division du compteur par 4 puis qu'il y a 4 leds
31     led=LED[I] #on va chercher la led dans la liste qui correspond au reste de la division
32     led.value(1) #on met cette led à 1
33     print("valeur =", val_new, I)
34     val_old = val_new    # on reinitialise val_old avec la valeur actuelle pour pouvoir détecter
35                           #un nouveau changement
36     time.sleep_ms(50)
37
38
39 if __name__ == '__main__':
40     while True:
41         lecture_encoder(ENCODER)
```

# G|1 Glossaire informatique1

**Instruction du langage** : ne nécessite pas d'importer de module pour les utiliser

**print('Bonjour')** : affiche Bonjour sur la console du PC

**While test** : Boucle tant que test est VRAI

**import** : pour importer des modules contenant des fonctions

**import time** : permet d'appeler les fonctions relatives au temps

**time.sleep( x )** : arrête l'exécution du programme pendant x secondes

**import machine** : module contenant les fonctions permettant d'agir sur les périphériques du pi pico

**from** : pour simplifier l'écriture on importe certaines fonctions directement dans une variable à l'aide de « from »

**from machine import Pin** : permet d'accéder à la fonction Pin du module machine uniquement.

**ma\_led\_verte = Pin( n, Pin.OUT )** : permet d'indiquer comment on souhaite utiliser un GPIO,  
ici on a configuré le p I/O N° n en sortie tout ou rien

**ma\_led\_verte.Toggle()** : inverse l'état de l'I/O associé à la variable.

Ici l'IO n passera de l'état haut (+3.3v) à l'état bas (0v) ou inversement

**#** : En début de ligne permet d'écrire un commentaire, ce qui est écrit n'est pas une instruction et donc n'est pas pris en compte par le programme.

**Variable** **A=A+1** -> Nouvelle valeur = ancienne valeur+1 , peut s'écrire **A+=1**

**print (" texte ", A), input ("texte", B)**

**Opérateurs**: +,-,\*,/,%,      **Données**: int(), float(), str(), bool(=True or False

# G|2 Glossaire informatique2

Liste -> Création d'une liste, `liste_de_noms = ['Toto', 'Hubert', 'Jacques', 'Philippe']`

`liste_quelconque = ['Toto', 1, 'Philippe', 4,5,6]`

`print ('Nombre d'articles dans la liste : ', len(liste_quelconque))`

`for items in liste_quelconque:`

`print (items)` -> parcourt la liste et imprime chaque item de la liste

Pour récupérer un élément de la liste, **variable= liste[i]**, i étant l'indice d'ordre dans la liste

Dictionnaire-> liste d'éléments sous la forme d'une paire key:valeur.

`N = {'voiture':'4 roues', 'moto':'2 roues'}`

On accède à la valeur d'un key en l'utilisant comme index (clé de recherche):

`print(N['voiture'])` -> '4 roues'

Fonction-> `def nom_de_ma_fonction(args) :`

on peut ensuite l'appeler dans un programme avec: `nom_de_ma_fonction()`

**Module**-> `from machine import Pin` -> importe la fonction Pin du module machine

`P0 = Pin(0, Pin.OUT)` -> on paramètre le GPIO 0 en sortie

`P0.value(1)` -> on lui donne la valeur 1 ou état « haut »



# G|3 Glossaire informatique 3

## Boucles

**for degree in range(0,180,1):**

**XXXXX**

-> la variable degree va varier de 0 à 180 avec un pas de 1, pour chaque valeur de cette variable, XXXX sera exécuté

**global ma\_variable** -> permet d'avoir accès à ma\_variable du programme principal dans la def d'une fonction

**def ma\_fonction():**

**global ma\_variable**

**print( ma\_variable)**

**ma\_variable += 2**

**return ma\_variable**

**Return ma\_variable** -> permet de donner au reste du programme la nouvelle valeur de ma\_variable ou d'une nouvelle variable si créée dans la fonction

# G|4 Les fonctions du module time

**time.time()** -> donne le temps présent en ms

**time.time\_diff(ticks1, ticks2)** -> donne la différence entre ticks1 et 2

**time.sleep (2)** -> met un délai de 2 secondes

**utime** travaille comme time mais en ms

**utime.sleep (100)** -> met un délai de 100ms

# G|5 Glossaire informatique 3

Les tests:

l'instruction **if...else** permet d'exécuter un bloc de code en fonction de la véracité d'une condition,

où le corps de l'instruction **if** s'exécute si la condition est **vraie**,

et le corps de l'instruction **else** s'exécute si elle est **fausse**,

avec une indentation pour définir les blocs de code.

Dans la suite des instructions du programme,

Exemple:

Dans le programme A=2

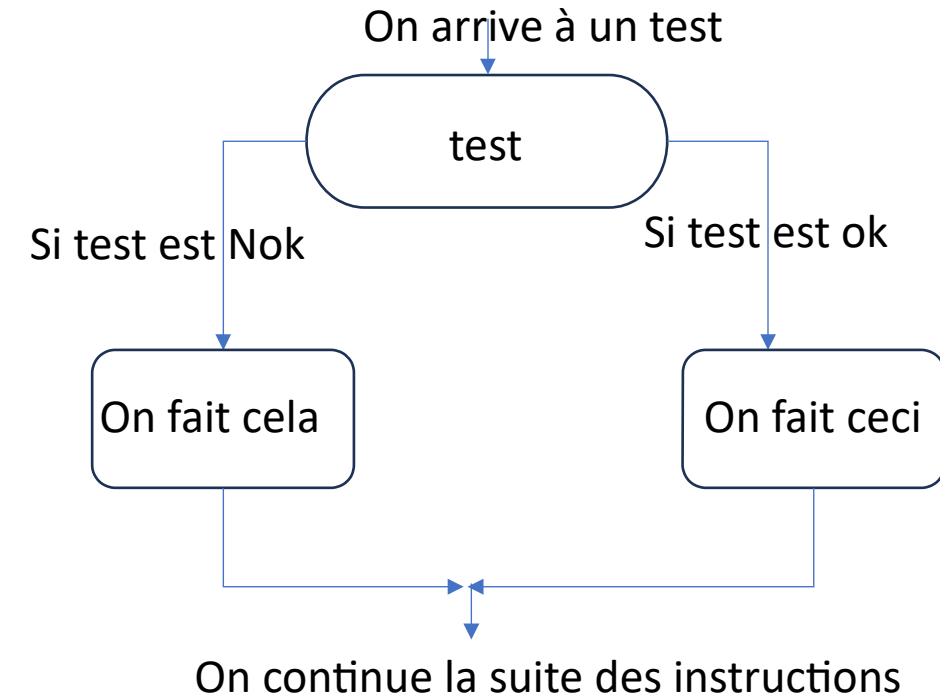
If A > 10:

    print('A supérieur à 10')

else:

    print ('A inférieur à 10')

On continue d'exécuter les instructions qui suivent



# Gl6 Les fonctions du module PWM

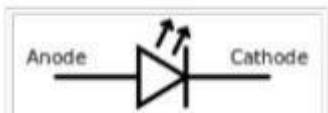
Fonctions utilisées:

```
servoPinPan = PWM(Pin(16))          # servoPinPan est un objet, initialise le Pin GPIO 16 en sortie PWM  
servoPinPan.freq(50)                # Initialise la fréquence des signaux carrés à 50Hz soit 20ms  
servoPinPan.duty_u16(int(newDuty))  # duty_u16 est le rapport cyclique entre le temps du signal carré haut sur  
sa période, new duty est une variable, le max est à 65535.
```

```
servoPinPan.deinit()                # réinitialise le PWM
```

```
servoX.duty_ns(pulseDX) #commande le rapport cyclique du PWM en nanosecondes suivant la variable  
pulseDX de l'objet servoX
```

# GE1 Glossaire électronique



Symbole de la diode LED



Pile



Résistance



BP

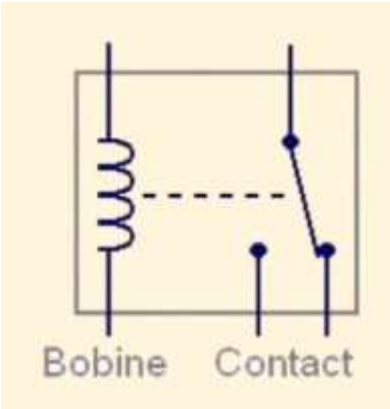
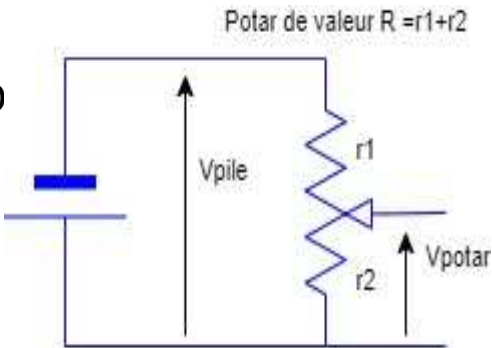


Schéma électrique du relais



$V_{potar} = V_{pile} * r2 / (r1 + r2)$



Condensateur  
de filtrage signal



Condensateur  
Chimique, réserve d'énergie

| Couleurs | Tension de seuil ou Vf | If (mA) | Longueur d' onde |
|----------|------------------------|---------|------------------|
| Rouge    | 1,6 V à 2 V            | 6à20    | 650 à 660 nm     |
| Jaune    | 1,8 V à 2 V            | 6à20    | 565 à 570 nm     |
| Vert     | 1,8 V à 2 V            | 6à20    | 585 à 590 nm     |
| Bleu     | 2,7 V à 3,2 V          | 6à20    | 470 nm           |
| blanc    | 3,5 v à 3,8 v          | 30      |                  |

